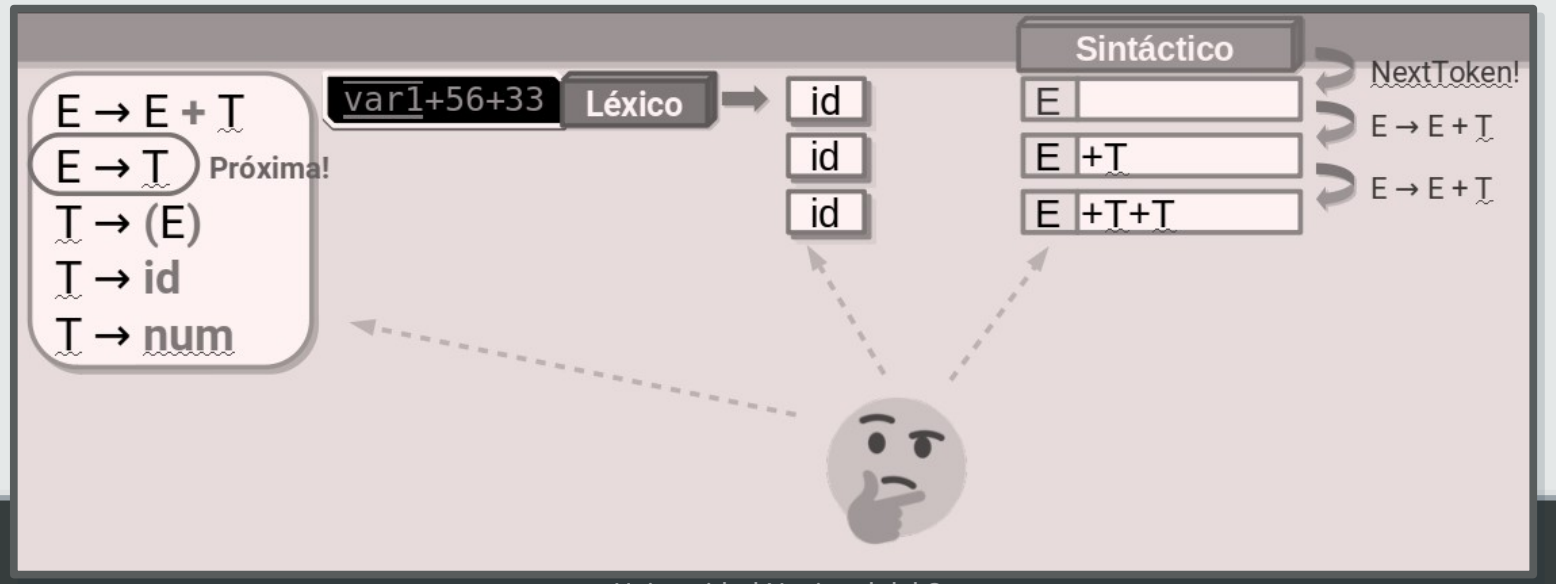
Compiladores e Intérpretes

Tema 3.2.3 – Analizador Sintáctico – Descendente – Ambüedad, Factorización y Recursión a Izquierda

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
2026

¿Como Elegir la producción a aplicar?

- La clave de la estrategia está en **cómo elegir las producciones** cuando estoy analizando un NT
- Claramente esto se tiene que hacer de forma **automatizada** y por eso hay varios **desafíos**… por ejemplo:



¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

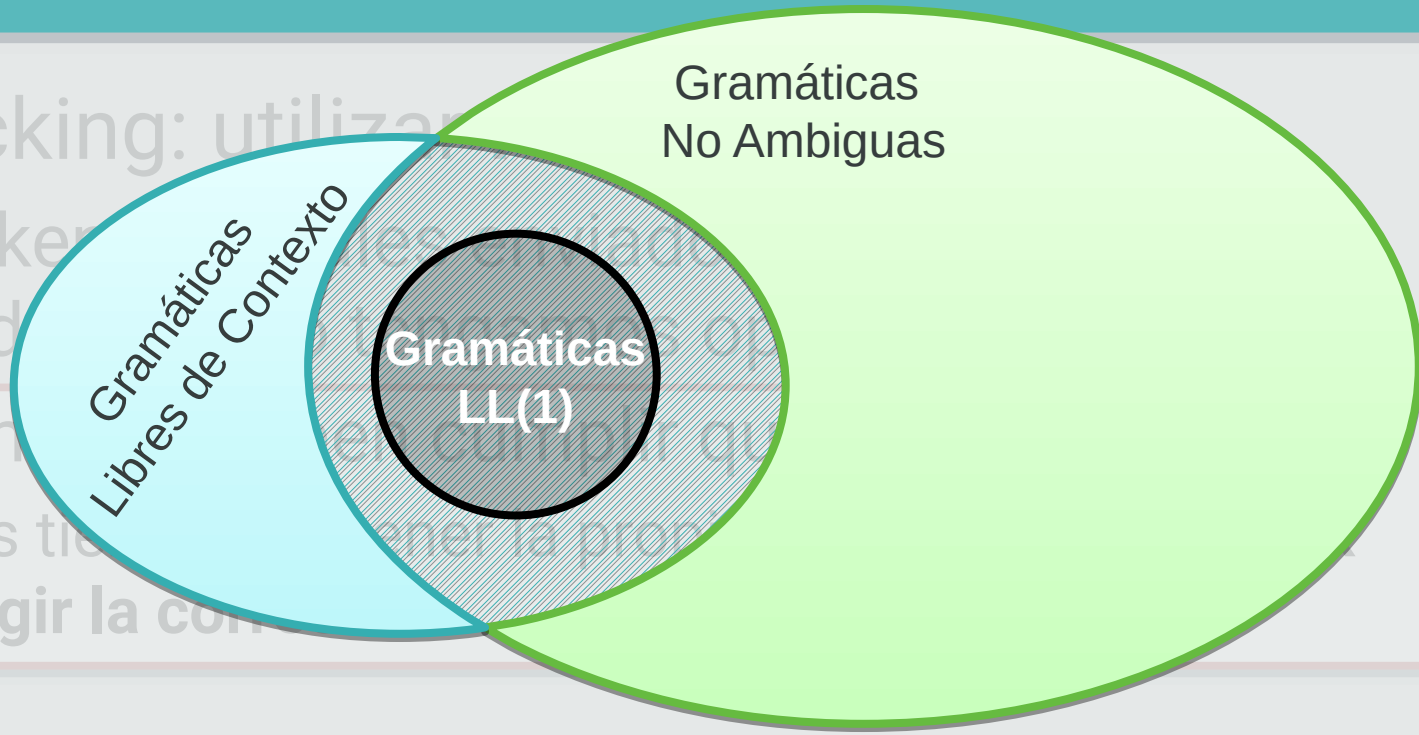
¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

Este tipo de gramáticas son llamadas **LL(k)**
L: cadena de entrada se procesa de izquierda a derecha (**L**eft scannig)
L: derivación a izquierda (**L**eftmost derivation)
k: cantidad de símbolos de predición

¿Como Elegir la producción a aplicar?

Nosotros vamos a trabajar con gramáticas **LL(1)**, es decir con 1 símbolo/token de predicción



Este tipo de gramáticas son llamadas **LL(k)**

L: cadena de entrada se procesa de izquierda a derecha (**L**eft scannig)

L: derivación a izquierda (**L**eftmost derivation)

k: cantidad de símbolos de predición

Primeros

β es una secuencia que puede tener terminales y no terminales

- En General:

  Consideremos una producción de la forma Y → β y un token **x**

Si a partir una forma sentencial β podemos llegar mediante derivación a otra forma donde el *terminal* **x** aparece primero decimos que:

x esta en Primeros(β)

Primeros



Intuitivamente Primeros(β) nos da el conjunto de todos los **terminales** que pueden aparecer **al principio** de las formas resultantes de **derivaciones de β**, Por ejemplo:

L → D L | **$**
D → A | M
A → V T **id**
M → T **id(**P**)**
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(**id**) = ?
Primeros(V) = ?
Primeros(VT**id**) = ?
Primeros(T**id(**P**)**) = ?
Primeros(A) = ?
Primeros(DL) = ?
Primeros(L) = ?

Primeros



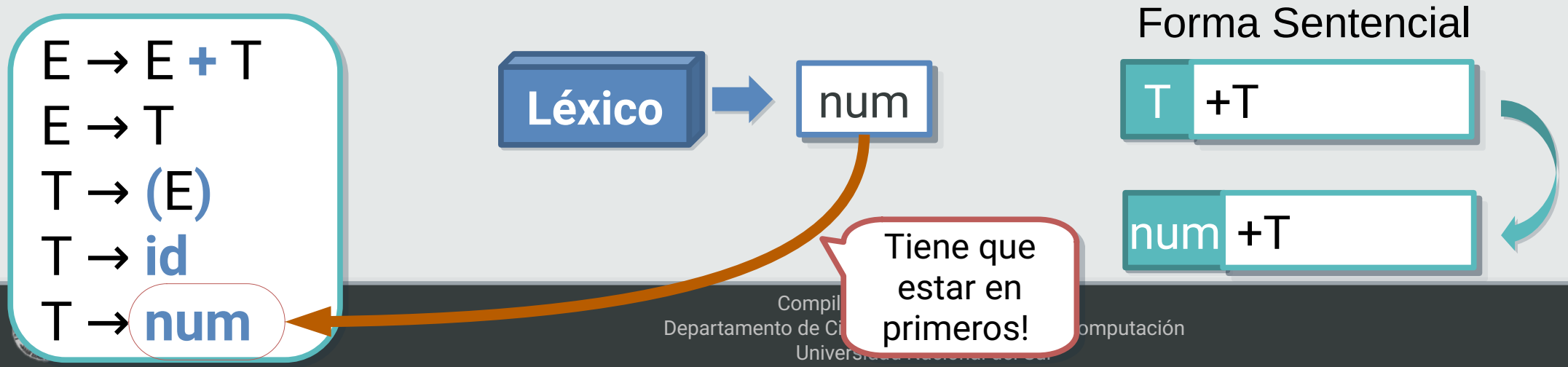
- Si por alguna razón β nos lleva a vacío (ε) por completo, entonces e esta en Primeros(β)

L → D L | **$**
D → A | M
A → V T **id**
M → T **id**(P)
V → **priv** | **pub**
T → **int** | **char**
P → T **id** P | ε

Primeros(ε) = ?
Primeros(P) = ?
Primeros(P**id**)= ?
Primeros(PT) = ?
Primeros(PP) = ?
Primeros(PV) = ?

Usando Primeros en la Estrategia Descendente

- Intuición: **Simular** la derivación a izquierda
  - Emular la forma sentencial de cada paso de la derivación, analizando el **símbolo** (T o NT) **de más a la izquierda** y el **token actual** enviado por el Léxico
  - Si el símbolo de más a la izquierda de la forma es:
    - **NT**: **elegir** una **producción** para ese no terminal **que tenga al token actual en primeros de su cuerpo** y **reemplazarlo por la parte derecha**



¿Como Elegir la producción a aplicar?

- Por ejemplo

T → (E) | **id** | **num**

- Para **T** siempre podemos elegir, si el **token actual** es:
  - (elegimos la primera
  - **Id** la segunda
  - **num** la tercera

¿Como Elegir la producción a aplicar?

- Por ejemplo

T → (E) | **id** | **num**

- Para **T** siempre podemos elegir, si el **token actual** es:
  - (elegimos la primera
  - **Id** la segunda
  - **num** la tercera

Básicamente, elegimos la **producción** que tenga al **token actual** en los **primeros** de su cuerpo

Primeros y LL(1)

- Cualquier gramática LL(1), para un NT con producciones de la forma

$$A \rightarrow \beta 1 \mid \beta 2 \mid \ldots \mid \beta n$$

- Verifica que, para cualquiera $i \neq j$:

$$Primeros(\beta i) \cap Primeros(\beta j) = \emptyset$$



¿Como Elegir la producción a aplicar?

- Alternativa para el backtracking: utilizar **Predicción**
  - La intuición es utilizar los tokens actuales enviados por el léxico para elegir la producción adecuada cuando tengamos opciones
  - Las producciones de la gramática deben cumplir que
    - Dado un NT sus producciones tienen que tener la propiedad de que al **mirar k tokens siempre podemos elegir la correcta**

Este tipo de gramáticas son llama
L: cadena de entrada se procesa de izqu
L: derivación a izquierda (**L**eftmost deriv
k: cantidad de símbolos de predición

Si la gramática **no cumple** con esto debemos **adaptarla sin cambiar** el lenguaje que genera

Factorización

¿Como Elegir la producción a aplicar?

- No son LL(1) por que tienen producciones al menos dos prods donde no sabemos cual elegir (intersección primero no vaciá):

T → (E) | **id** | **id**(E) | **num**

En T si el token actual es **id** no sé si elegir la 2da o la 3ra

S → A | L
A → **id** = E | **num**
L → **id**(E)

En S si el token actual es **id** no sé si elegir la 1ra o la 2da

¿Como Elegir la producción a aplicar?

- No son LL(1) por que tienen producciones al menos dos prods donde no sabemos cual elegir (intersección primero no vaciá):

T → (E) | **id** | **id**(E) | **num**

En T si el token actual es **id** no sé si elegir la 2da o la 3ra

S → A | L
A → **id** = E | **num**
L → **id**(E)

En S si el token actual es **id** no sé si elegir la 1ra o la 2da

Problema: producciones con prefijo izquierdo común (o que llevan a un prefijo izquierdo común)

Se puede solucionar **factorizando a izquierda**

Factorización a Izquierda

- Factorización a Izquierda Directa:

$T \rightarrow (E) \mid \mathbf{id} \mid \mathbf{id}(E) \mid \mathbf{num}$



Factorización a Izquierda

- Factorización a Izquierda Directa:
  - **Intuición:** Para cada no terminal X
    - Hacer factor común de producciones que **comparten prefijos directamente**

      $X \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma_1 \mid \ldots \mid \gamma_m$
    - **Transformar en:**

      $X \rightarrow \alpha R \mid \gamma_1 \mid \ldots \mid \gamma_m$

      $R \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$



Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio

Factorización a Izquierda

- Factorización a Izquierda Directa:
  - **Intuición:** Para cada no terminal X
    - Hacer factor común de producciones que **comparten prefijos directamente**

      $X \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \ldots \mid \alpha\beta_n \mid \gamma_1 \mid \ldots \mid \gamma_m$
    - **Transformar en:**

      $X \rightarrow \alpha R \mid \gamma_1 \mid \ldots \mid \gamma_m$

      $R \rightarrow \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

α Es el prefijo común



Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio

Factorización a Izquierda

- Factorización a Izquierda Profunda:
  - **Intuición:** Para cada no terminal X
    - Si el prefijo no es directo, el proceso es más manual…
    - **Tratar** de “subir” las producciones que llevan al prefijo común hasta NT, y después hacer factor común

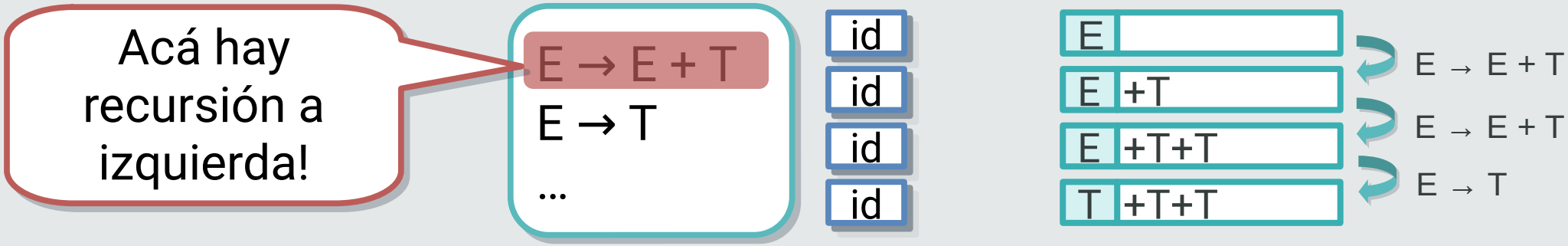
S → A | L
A → **id =** E | **num**
L → **id(**E**)**



Eliminación de Recursión a Izquierda

¿Como Elegir la producción a aplicar?

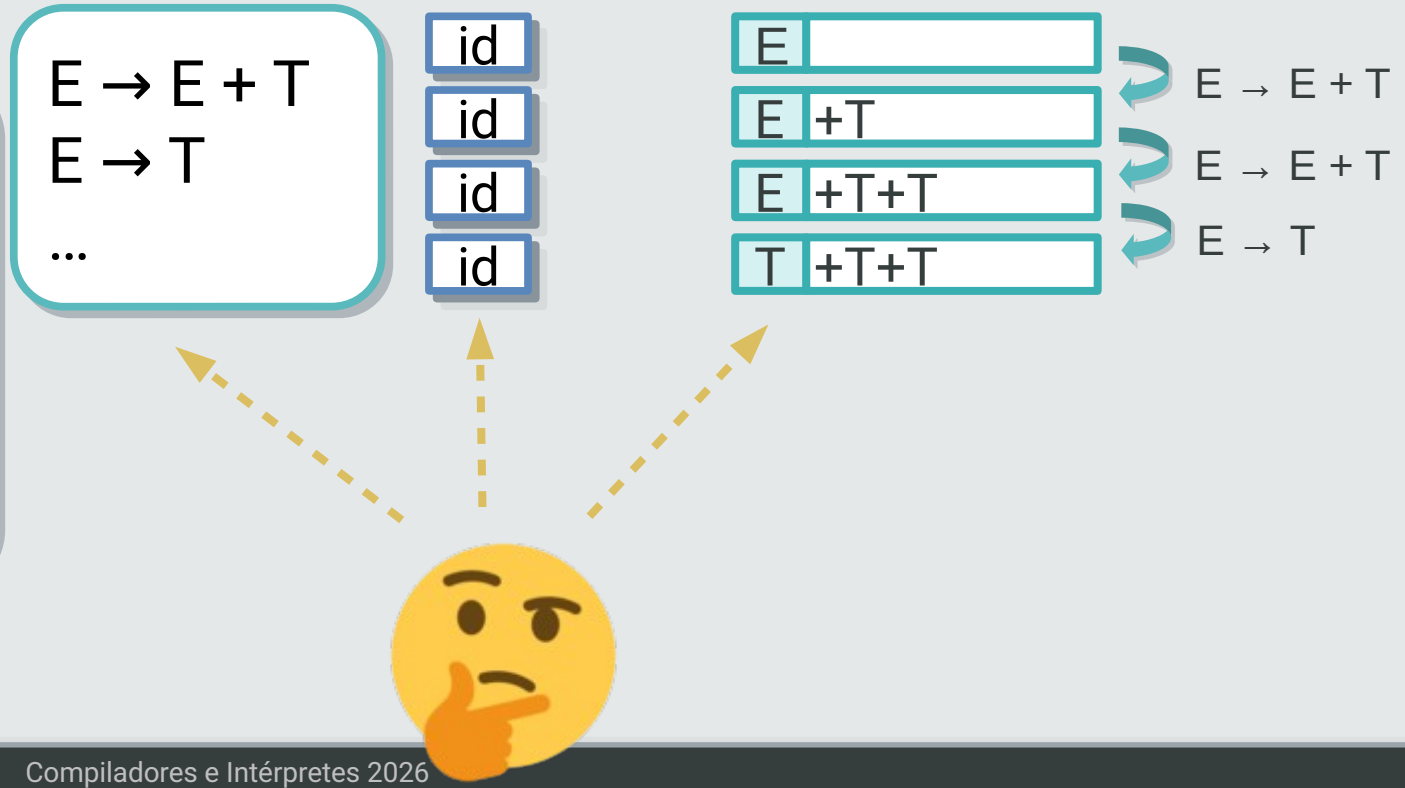
- Si la gramática tiene **recursión a izquierda → no se cuando cortar la recursión!**



¿Como Elegir la producción a aplicar?

- Si la gramática tiene **recursión a izquierda → no se cuando cortar la recursión!**

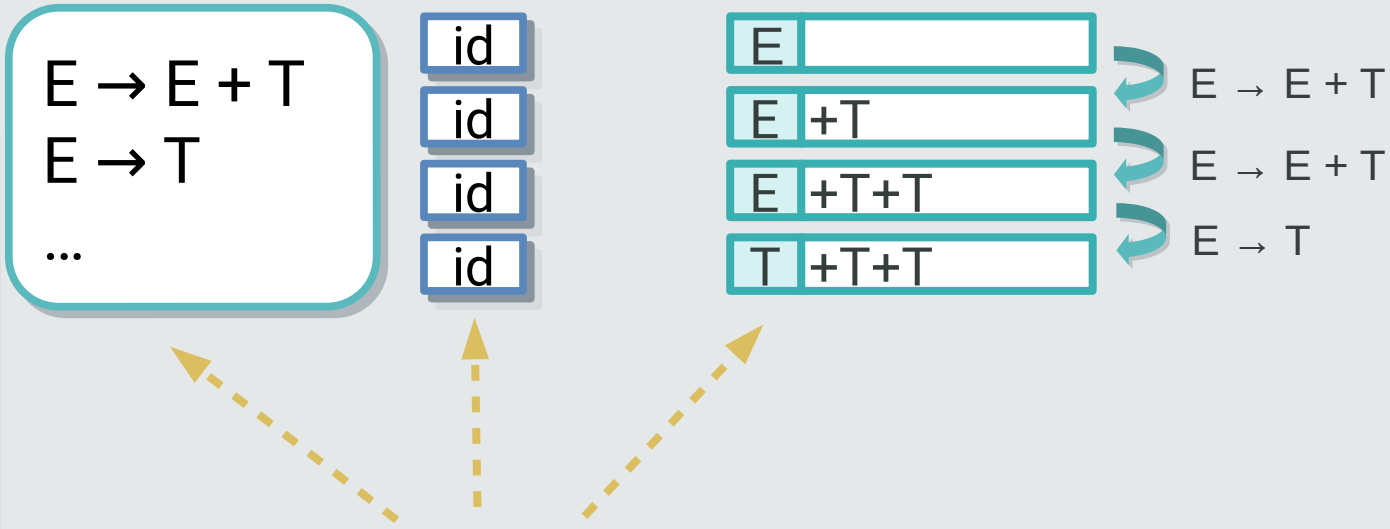
Este es el problema que habíamos visto antes, básicamente con id y E puedo aplicar tanto la primer como la segunda regla… **Eso es por que la gramática no es LL(1)**



¿Como Elegir la producción a aplicar?

- Si la gramática tiene **recursión a izquierda → no se cuando cortar la recursión!**

Este es el problema que habíamos visto antes, básicamente con id y E puedo aplicar tanto la primer como la segunda regla… **Eso es por que la gramática no es LL(1)**

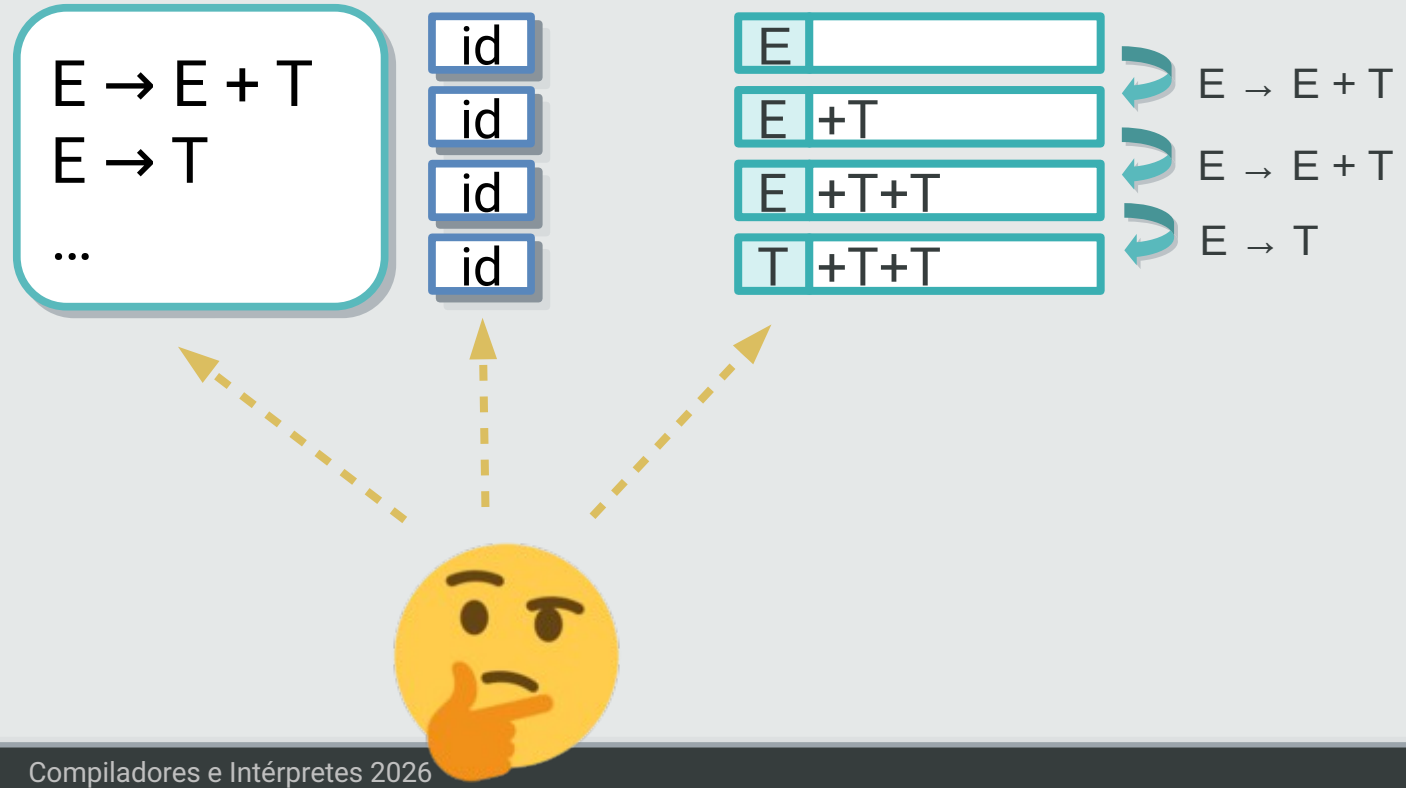


Ademas, si miramos **primeros(E+T)** y **primeros(T)** ambos contienen **id**… por lo tanto no se que producción elegir

¿Como Elegir la producción a aplicar?

- Si la gramática tiene **recursión a izquierda → no se cuando cortar la recursión!**

La recursión a izquierda se puede eliminar reescribiendo la gramática!



Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

En la derivación se puede ver ese comportamiento de las produciones con recursion a izquierda

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

Para eliminar la recursión a izquierda la idea es invertir la forma en que vamos armando la cadena… Sabiendo que va arrancar con algun beta y despues se pueden repetir muchos alfas o terminar

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

Para eliminar la recursión a izquierda la idea es invertir la forma en que vamos armando la cadena… Sabiendo que va arrancar con algun beta y despues se pueden repetir muchos alfas o terminar

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

$\Rightarrow y\ldots\ yz\ldots\ yzz\ldots\ yzzz\ldots$

Para eliminar la recursión a izquierda la idea es invertir la forma en que vamos armando la cadena…
Sabiendo que va arrancar con **y** y despues se pueden repetir muchas **z** o terminar

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

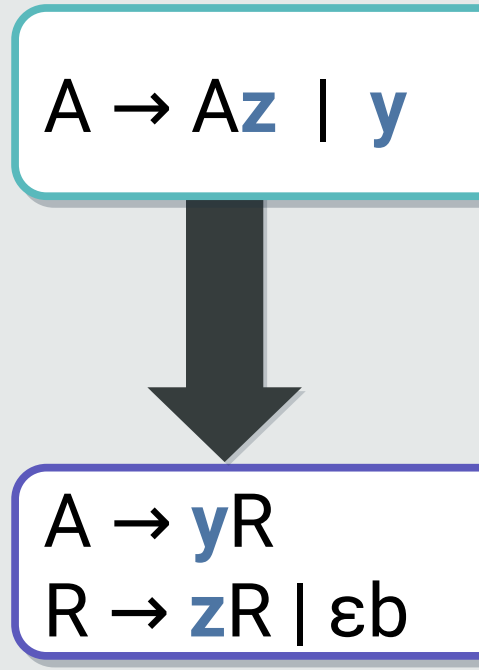
$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

$\Rightarrow yR \Rightarrow yzR \Rightarrow yzzR \Rightarrow yzzzR...$

Para eliminar la recursión a izquierda la idea es invertir la forma en que vamos armando la cadena… Sabiendo que va arrancar con **y** y despues se pueden repetir muchas **z** o terminar

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:



Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

$A \rightarrow yR$
$R \rightarrow zR \mid \varepsilon$

$A \Rightarrow yR \Rightarrow yzR \Rightarrow yzzR \Rightarrow yzzzR \Rightarrow yzzz$

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:
  - $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$
  - Transforma en:
    - $A \rightarrow \beta_1 R \mid \beta_2 R \mid \ldots \mid \beta_m R$
    - $R \rightarrow \alpha_1 R \mid \alpha_2 R \mid \ldots \mid \alpha_n R \mid \varepsilon$

$A \rightarrow Az \mid y$

$A \rightarrow yR$
$R \rightarrow zR \mid \varepsilon b$



Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:
  - $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$
  - Transforma en:
    - $A \rightarrow \beta_1 R \mid \beta_2 R \mid \ldots \mid \beta_m R$
    - $R \rightarrow \alpha_1 R \mid \alpha_2 R \mid \ldots \mid \alpha_n R \mid \varepsilon$

Miremos en detalle las reglas de A...
Las primeras nos permiten ir agregando αs al final de la cadena



Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio

Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:
  - $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$
  - Transforma en:
    - $A \rightarrow \beta_1 R \mid \beta_2 R \mid \ldots \mid \beta_m R$
    - $R \rightarrow \alpha_1 R \mid \alpha_2 R \mid \ldots \mid \alpha_n R \mid \varepsilon$

$A \rightarrow Az \mid y$

$A \Rightarrow Az \Rightarrow Azz \Rightarrow Azz \Rightarrow yzzz$

Para eliminar la recursión a izquierda la idea es invertir la forma en que vamos armando la cadena… Sabiendo que va arrancar con algun beta y despues se pueden repetir muchos alfas o terminar

Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio



Eliminar la Recursión a Izquierda

- Eliminando la recursión a izquierda
- Directa:
  - $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$
  - Transforma en:
    - $A \rightarrow \beta_1 R \mid \beta_2 R \mid \ldots \mid \beta_m R$
    - $R \rightarrow \alpha_1 R \mid \alpha_2 R \mid \ldots \mid \alpha_n R \mid \varepsilon$

$E \rightarrow E + T \mid T$
$T \rightarrow (E) \mid id \mid num$





Las letras griegas α_i y β_i representan secuencias de T y/o NT o vacio

Analizadores Sintácticos Descendentes

- Si partimos de una gramática no factorizada y con recursión a izquierda, es importante **primero eliminar la recursión y luego factorizar**
  - ¿Por qué?

Analizadores Sintácticos Descendentes

- Si partimos de una gramática no factorizada y con recursión a izquierda, es importante **primero eliminar la recursión y luego factorizar**
  - ¿Por qué?

Al eliminar la recursión a izquierda podemos generar nuevas producciones no factorizadas

Analizadores Sintácticos Descendentes

- Si partimos de una gramática no factorizada y con recursión a izquierda, es importante **primero eliminar la recursión y luego factorizar**
  - ¿Por qué?

Al eliminar la recursión a izquierda podemos generar nuevas producciones no factorizadas

- **Factorizar de más** (especialmente cuando el prefijo no es directo) puede ser negativo para el **compilador**
  - ¿Por qué?

Analizadores Sintácticos Descendentes

- Si partimos de una gramática no factorizada y con recursión a izquierda, es importante **primero eliminar la recursión y luego factorizar**
  - ¿Por qué?

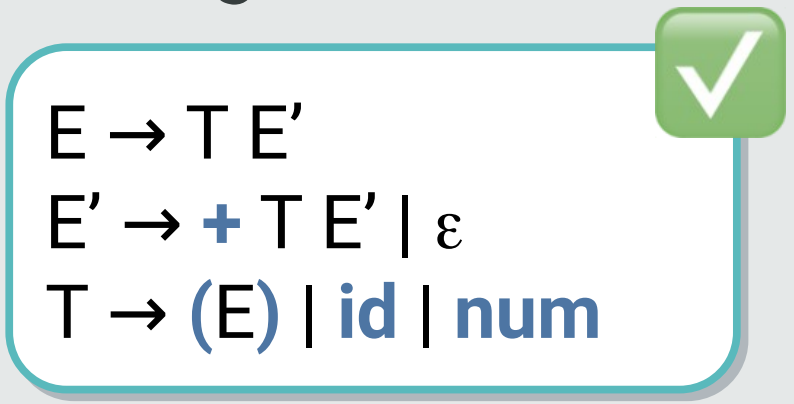
Al eliminar la recursión a izquierda podemos generar nuevas producciones no factorizadas

- **Factorizar de más** (especialmente cuando el prefijo no es directo) puede ser negativo para el **compilador**
  - ¿Por qué?

Muchas veces las reglas de la gramática original tienen un significado, al modificarla al analizador semántico le puede llegar a ser más difícil entender el programa

Analizadores Sintácticos Descendentes

- Una vez que podemos asegurar que nuestra gramática:
  - Es Libre de contexto
  - No es ambigua (o es arreglable)
  - No tiene recursión a izquierda
  - Está factorizada a izquierda



Podemos Construir un Analizador Sintáctico Descendente Predictivo

¿Como Seguimos?

- Hay **dos estrategias** para **implementar** este tipo de analizadores sintáctico
  - **Recursivos**
  - **Basados en Tabla**

La etapa 2 del proyecto se basará en esta estrategia!

- En los temas que siguen exploraremos en detalle estas estrategias!

El Problema de la Ambigüedad

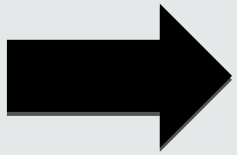
Derivación

- Cadena derivable: partiendo del símbolo inicial I aplicamos producciones y llegamos a la cadena

Una **cadena derivable** en una gramática G pertenece al **lenguaje** de G.

Aplicar producción: reemplazar NT por la parte derecha de una producción que lo tenga como lado izquierdo.

Expresiones Básicas
E → E + E
E → (E)
E → id
E → num



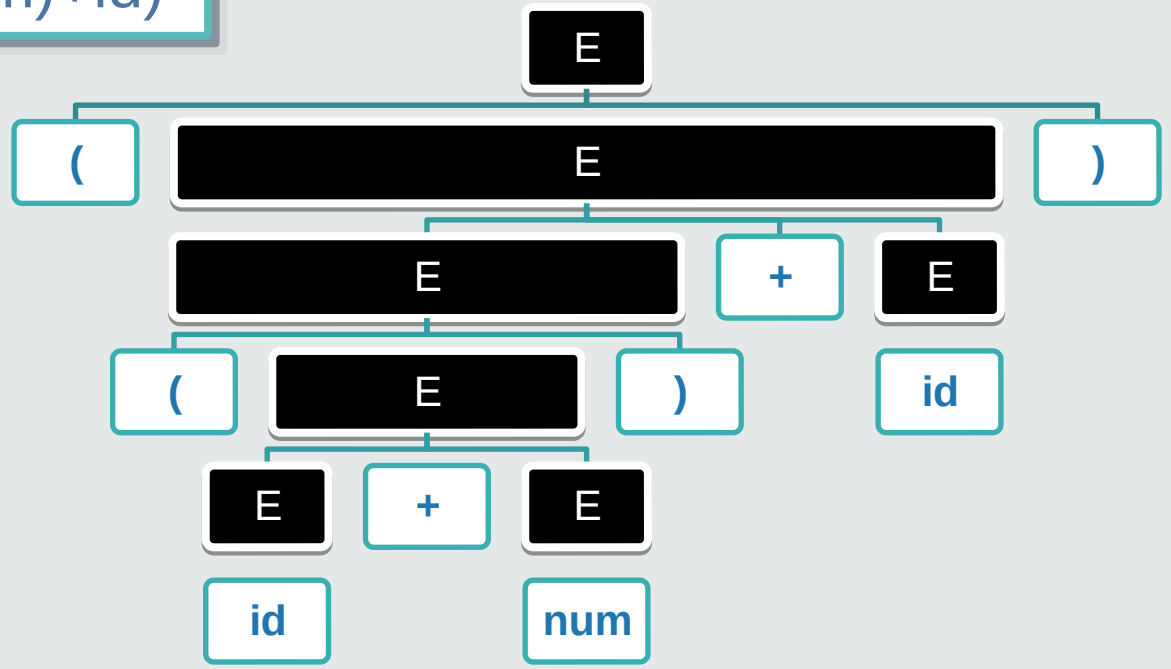
((id+num)+id)

Arbol Sintáctico

- Una derivación puede verse como un **Árbol Sintáctico**

Expresiones Básicas
E → E + E
E → (E)
E → id
E → num

((id+num)+id)



Terminales son **hojas** del árbol
No terminales son **nodos internos**
Símbolo Inicial es la **raíz**

Ambigüedad

- Una **gramática** es **ambigua** cuando para una misma **cadena** existen al menos **dos arboles de derivación diferentes**

E → E + E
E → E * E
E → id
E → num



id+id+id

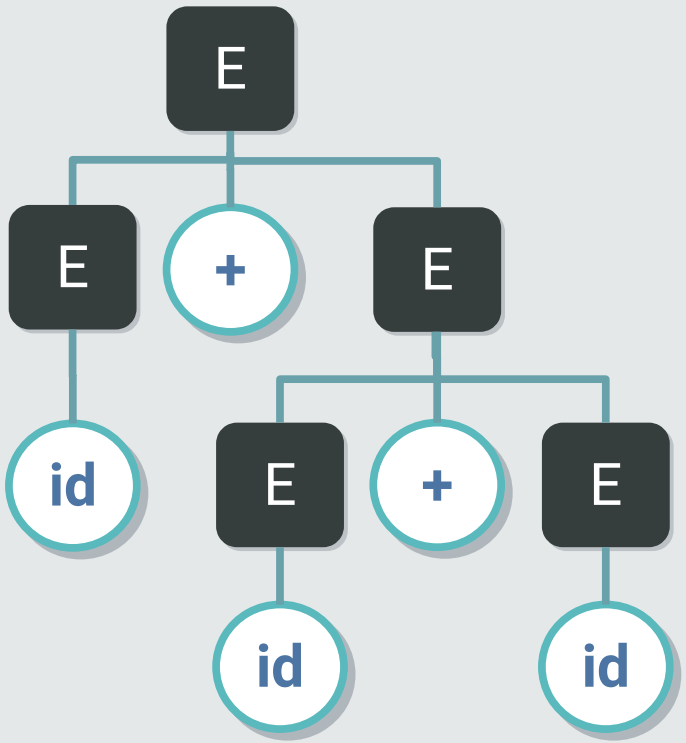


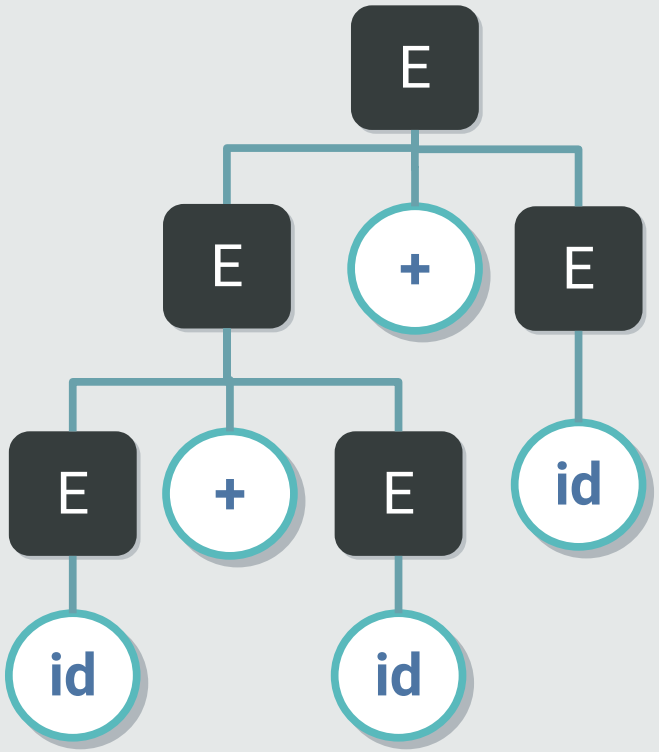
Ambigüedad

- Una **gramática** es **ambigua** cuando para una misma **cadena** existen al menos **dos arboles de derivación diferentes**

E → E + E
E → E * E
E → id
E → num

id+id+id





Ambigüedad - Problemas

La **ambigüedad** en las reglas de sintaxis de un lenguajes trae **problemas** a la hora de **implementar su compilador**

- Dificulta las estrategias de análisis sintáctico!
  - Hay que dotar al **analizador sintáctico** de **mecanismos especiales** para decidir entre alternativas sintácticamente equivalentes
- **Problema Semántico:** dificulta el entendimiento del significado del programa

Ambigüedad

- **Soluciones**: (no siempre posible!)
  - **Reescribir la gramática** en una gramática no ambigua que genere el mismo lenguaje

$$E \rightarrow E + E \mid E * E \mid id \mid num$$



$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow id \mid num$$

  - Agregar **reglas especiales** (precedencia / asociatividad) en la implementación del analizador sintáctico